Complete Project Explanation: File System Simulator

> *This document explains **everything** about this project as if you know nothing about it. Read this top to bottom and you'll understand every file, every folder, and exactly what happens when you run it.*

What Is This Project?

This project **pretends to be a hard disk and a file system**. Just like your Windows computer has a C: drive with folders and files, this project creates a **fake disk** (a 10 MB file) and lets you create folders and files inside it using a terminal.

The magic is: you can **crash** this fake disk (simulate a power failure), and then **recover** your files — demonstrating how real operating systems like Windows (NTFS) and Linux (ext4) protect your data.

Project Folder Structure

Here is everything inside `c:\Ashwell\Project\OS Project\` :

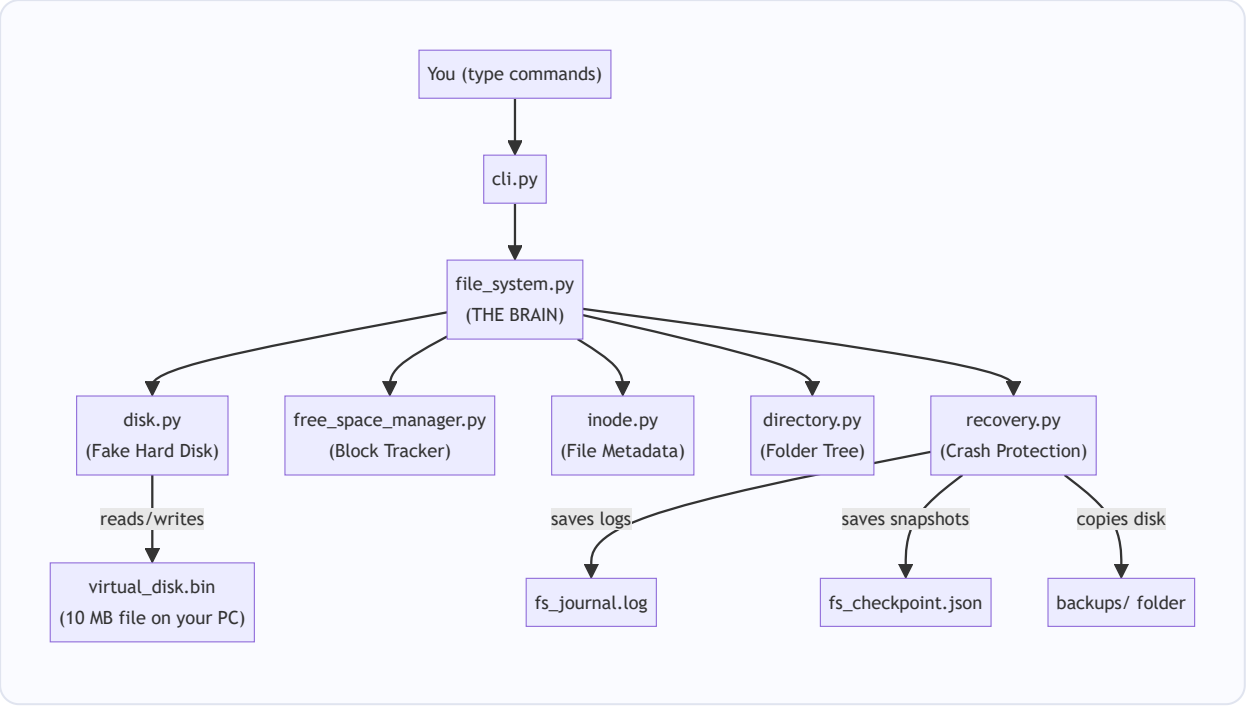
```
c:\Ashwell\Project\OS Project\
|
|-- disk.py                  ⟵ SOURCE CODE (the fake hard disk)
|-- free_space_manager.py    ⟵ SOURCE CODE (tracks which blocks are free/used)
|-- inode.py                 ⟵ SOURCE CODE (file metadata storage)
|-- directory.py             ⟵ SOURCE CODE (folder tree structure)
|-- file_system.py           ⟵ SOURCE CODE (the brain - connects everything)
|-- recovery.py              ⟵ SOURCE CODE (journaling + checkpoint + backup)
|-- cli.py                   ⟵ SOURCE CODE (the terminal you type commands in)
|-- test_simulation.py       ⟵ SOURCE CODE (automated demo script)
|
|-- virtual_disk.bin         ⟵ GENERATED: The fake 10 MB hard disk file
|-- fs_journal.log           ⟵ GENERATED: Recovery journal (write-ahead log)
|-- fs_checkpoint.json       ⟵ GENERATED: Metadata snapshot for recovery
|-- backups\                 ⟵ GENERATED FOLDER: Stores full disk backup copies
|   |-- backup_20260412_demo.bin
```

```
|    |-- backup_20260412_demo.bin_fs_checkpoint.json
|    |-- backup_20260412_demo.bin_fs_journal.log
```

Part 1: SOURCE CODE FILES (What You Wrote)

These are the 8 Python files that make up the simulator. You never modify these — they ARE the program.

Flow: How They Connect



File 1: `disk.py` — The Fake Hard Disk

What it does: Creates and manages a file called `virtual_disk.bin` on your computer. This file IS the fake disk.

Think of it as: A hard disk drive (HDD) inside your computer.

Key details:
- The disk is exactly **10 MB** (10,485,760 bytes)
- It is divided into **2,560 blocks**, each block is **4 KB** (4,096 bytes)
- Just like a real disk has sectors, our disk has blocks

```
virtual_disk.bin (10 MB)
|---Block 0---|---Block 1---|---Block 2---|---  ...  ---|---Block 2559--|
|   4096 B    |   4096 B    |   4096 B    |             |    4096 B     |
```

It also contains:
- **LRU Cache** — Keeps recently-read blocks in computer memory (RAM) so the next read is instant. Like how your browser caches webpages.
- **Performance Tracker** — Times every read/write operation in milliseconds
- **Crash Buttons** — Functions to simulate power loss, corrupt blocks, or do partial writes

---

File 2: `free_space_manager.py` — Block Tracker

What it does: Keeps track of which blocks on the disk are FREE (empty) and which are USED (have data).

Think of it as: A parking lot map — showing which parking spots (blocks) are taken and which are available.

Two methods implemented:

Method A — Bitmap:

```
Block:  0   1   2   3   4   5   6   7   8   ...
Bit:    1   1   1   1   1   0   0   0   0   ...
        ^-- USED (has data)  ^-- FREE (available)
```

One bit per block. 0 = free, 1 = used. This is what our system uses.

Method B — Linked List:

```
Free List: [5] → [6] → [7] → [8] → [9] → ...
```

A list of free block numbers. Pop from the front to allocate.

File 3: `inode.py` — File Metadata

What it does: Every file and folder gets an **inode** (index node). The inode does NOT store the file content — it stores **information about the file**.

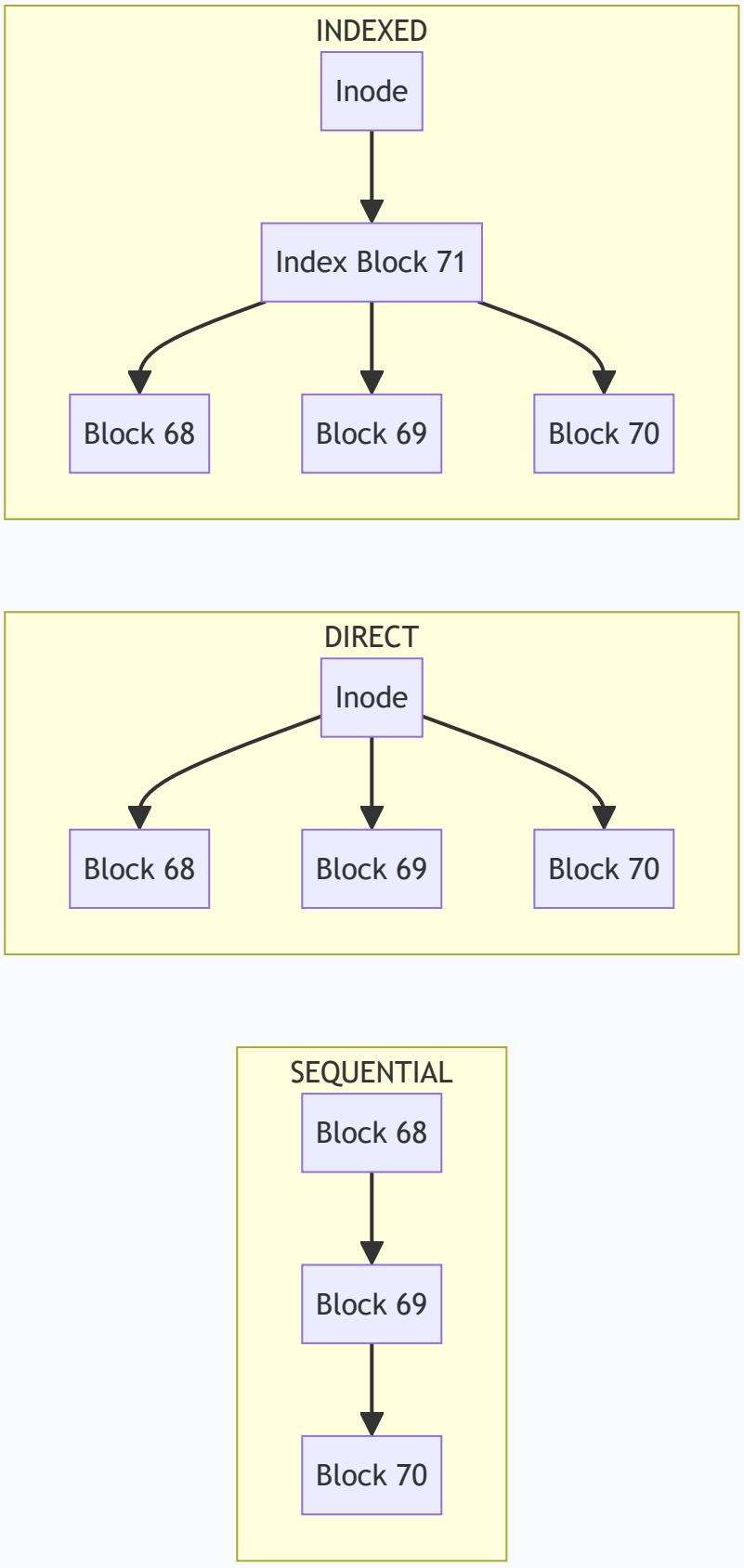
Think of it as: A library card catalog. The card tells you the book's title, author, and shelf location — but isn't the book itself.

What each inode stores:

```
Inode #5:
  - inode_id: 5
  - type: FILE (could be FILE or DIRECTORY)
  - size: 77 bytes
  - created: Sun Apr 12 16:05:40 2026
  - modified: Sun Apr 12 16:06:13 2026
  - access_method: SEQUENTIAL
```

```
  - block_pointers: [68]      ⟵ "Your data is in block 68"
  - index_block: -1           ⟵ not used for sequential
```

Three access methods:



Method	How It Works	Real-World Use
Sequential	Read blocks in order, one after another	Log files, video streaming
Direct	Inode directly points to each block	Small files, fast random access
Indexed	Inode points to an INDEX block, which points to data blocks	Large files (used in UNIX/Linux)

File 4: `directory.py` — Folder Tree

What it does: Maps folder paths (like `/documents/readme.txt`) to inode numbers. Creates the familiar folder hierarchy.

Think of it as: The folder structure you see in Windows Explorer.

```
/  (root directory = inode 0)
|
+-- documents/  (inode 1)
|   +-- reports/  (inode 2)
|   |   +-- q1.csv  (inode 6)
|   +-- readme.txt  (inode 5)
|
+-- images/  (inode 3)
|   +-- logo.dat  (inode 7)
|
+-- logs/  (inode 4)
```

Operations:
- `create` — Add a new file/folder entry
- `delete` — Remove an entry (and free its blocks)
- `rename` — Change the name of an entry
- `search` — Find a file by name across all folders
- `resolve_path` — Convert `/documents/readme.txt` to `inode 5`

File 5: `file_system.py` — THE BRAIN

What it does: This is the **central controller** that connects all other modules. When you type a command, this file decides what to do.

Think of it as: The operating system kernel. It talks to the disk, the block tracker, the inodes, the folders, and the recovery system.

Disk Layout — How the 10 MB disk is organized:

```
|  Block 0  | Blocks 1-3 | Blocks 4-67 | Blocks 68-2559 |
| SUPERBLOCK|   BITMAP   | INODE TABLE |   DATA BLOCKS  |
| (config)  | (free/used)| (file info) | (actual files) |
```

Region	Blocks	Purpose
Superblock	Block 0	Magic number "FSIM", version, disk size
Bitmap	Blocks 1-3	Free space tracker (2560 bits for 2560 blocks)
Inode Table	Blocks 4-67	Up to 1024 file/folder metadata records
Data Blocks	Blocks 68-2559	Where actual file content is stored

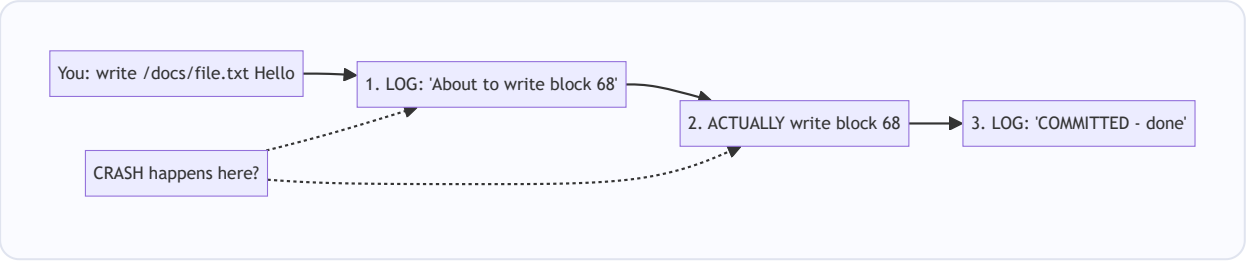
---

File 6: `recovery.py` — Crash Protection

What it does: Three layers of protection against data loss.

Layer 1: Write-Ahead Journal (`fs_journal.log`)

Before any change is made to the disk, the system writes a LOG entry saying "I'm about to do X". If a crash happens mid-operation, the log tells recovery what was in progress.



- If crash happens **before step 2**: Nothing was written. Journal says "uncommitted" — recovery knows to redo it.
- If crash happens **after step 3**: Everything is fine, operation completed.

Layer 2: Checkpoint (`fs_checkpoint.json`)

A full snapshot of all metadata saved as a JSON file. Contains:
- All inodes (file metadata)
- The entire directory tree
- The bitmap (which blocks are used)

When you run `recover`, the system loads this snapshot to restore the file system state.

Layer 3: Backup (`backups/` folder)

A complete copy of the entire `virtual_disk.bin` file. You can restore the whole disk to any previous backup.

File 7: `cli.py` — The Terminal Interface

What it does: The command prompt you interact with. Type commands, get results.

Think of it as: Command Prompt (cmd) or PowerShell, but for our fake file system.

When you run `python cli.py`, you see:

```
╔═══════════════════════════════════════════════════════╗
║       File System Recovery & Optimization Simulator         ║
║                     Interactive CLI                         ║
╚═══════════════════════════════════════════════════════╝

Type 'help' for a list of commands.

fs>
```

File 8: `test_simulation.py` — Automated Demo

What it does: Runs 13 pre-programmed steps automatically. Useful if you don't want to type commands manually.

Part 2: GENERATED FILES (Created When You Run The Program)

These files are NOT code. They are created automatically when the program runs.

`virtual_disk.bin` (10 MB)

Location: `c:\Ashwell\Project\OS Project\virtual_disk.bin`

What it is: THE fake hard disk. All your simulated files live inside this one binary file.

When you type `write /documents/readme.txt Hello`, the text "Hello" is stored somewhere inside this 10 MB file — specifically in one of the 4 KB blocks (block 68 onwards).

You can NOT open this file in Notepad — it's raw binary data.

Created when: You run `format` command.

`fs_journal.log`

Location: `c:\Ashwell\Project\OS Project\fs_journal.log`

What it is: A text file where every operation is logged BEFORE it happens.

Example contents:

```
{"txn_id": 1, "op_type": "MKDIR", "payload": {"path": "/documents"}, "committed":
{"txn_id": 2, "op_type": "CREATE_FILE", "payload": {"path": "/documents/readme.tx
{"txn_id": 3, "op_type": "WRITE_FILE", "payload": {"path": "/documents/readme.txt"
```

If `committed` is `true` — the operation completed successfully.
If `committed` is `false` — the operation was interrupted (crash!) and needs to be replayed.

Created when: First file operation after `format`.

`fs_checkpoint.json`

Location: `c:\Ashwell\Project\OS Project\fs_checkpoint.json`

What it is: A JSON snapshot of the entire file system metadata (NOT the file data itself, just the structure).

Contains:
- All inodes (which files exist, their sizes, which blocks they use)
- The full directory tree (which folders exist, what's inside them)
- The bitmap state (which blocks are free/used)

Used when: Recovery loads this to rebuild the file system after a crash.

Created when: You run `checkpoint`, `format`, or `defrag`.

`backups/` folder

Location: `c:\Ashwell\Project\OS Project\backups\`

What it is: Contains full copies of the virtual disk and its metadata files.

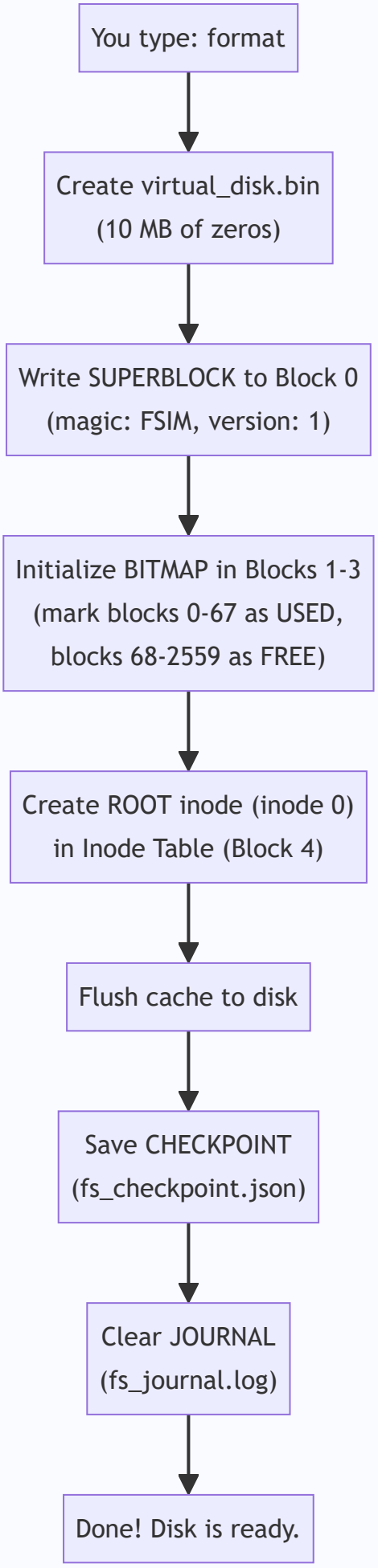
Each backup creates 3 files:

```
backups/
  backup_20260412_160907_demo.bin                    ⟵ Copy of virtual_disk.bin
  backup_20260412_160907_demo.bin_fs_checkpoint.json ⟵ Copy of checkpoint
  backup_20260412_160907_demo.bin_fs_journal.log     ⟵ Copy of journal
```

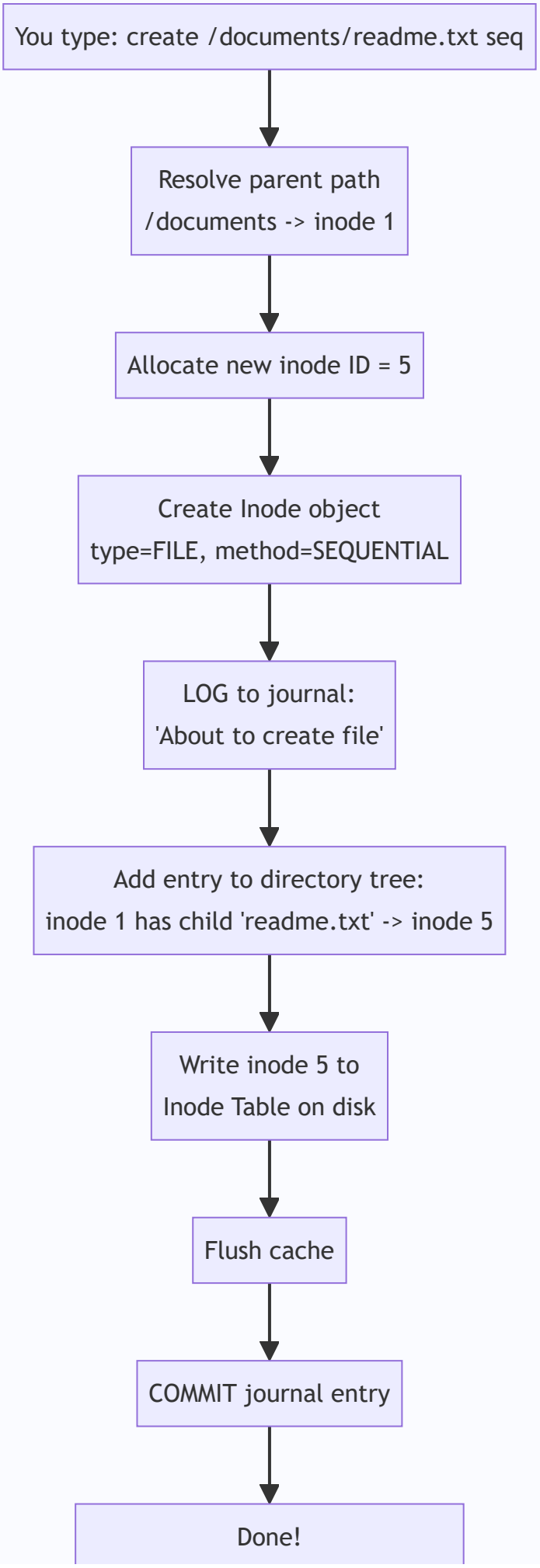
Used when: You run `restore <backup_name>` to go back in time.

Part 3: COMPLETE FLOW OF EVERY MAJOR OPERATION

Flow 1: `format` — Creating a New File System

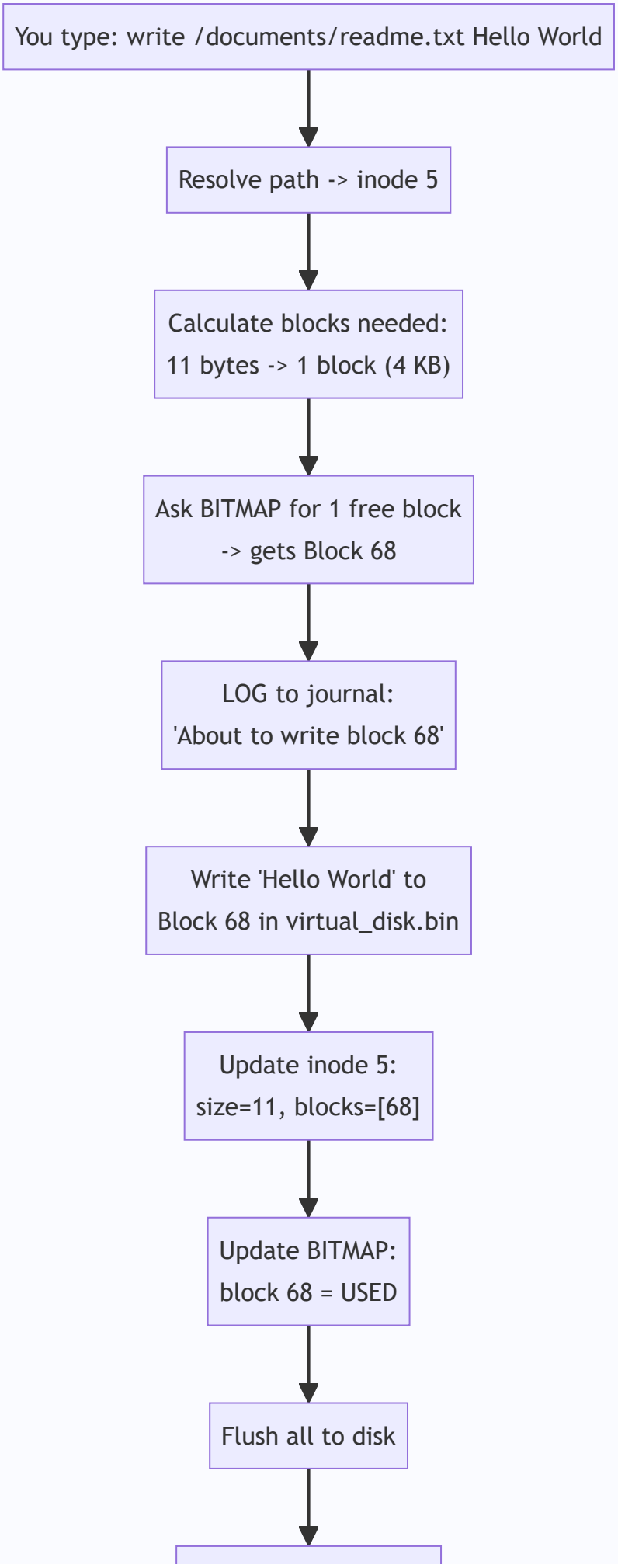

You type: format
Create virtual_disk.bin (10 MB of zeros)
Write SUPERBLOCK to Block 0 (magic: FSIM, version: 1)
Initialize BITMAP in Blocks 1-3 (mark blocks 0-67 as USED, blocks 68-2559 as FREE)
Create ROOT inode (inode 0) in Inode Table (Block 4)
Flush cache to disk
Save CHECKPOINT (fs_checkpoint.json)
Clear JOURNAL (fs_journal.log)
Done! Disk is ready.

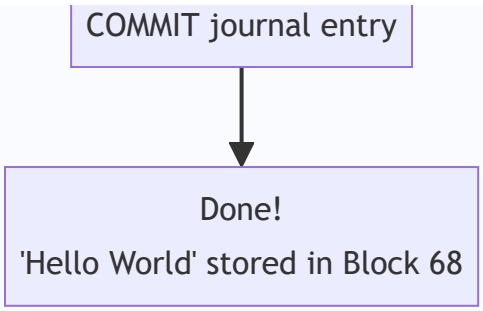
Flow 2: `create /documents/readme.txt seq` — Creating a File


You type: create /documents/readme.txt seq
Resolve parent path
/documents -> inode 1
Allocate new inode ID = 5
Create Inode object
type=FILE, method=SEQUENTIAL
LOG to journal:
'About to create file'
Add entry to directory tree:
inode 1 has child 'readme.txt' -> inode 5
Write inode 5 to
Inode Table on disk
Flush cache
COMMIT journal entry
Done!

File exists but is empty (0 bytes)

Flow 3: `write /documents/readme.txt Hello World` — Writing Data


You type: write /documents/readme.txt Hello World
Resolve path -> inode 5
Calculate blocks needed:
11 bytes -> 1 block (4 KB)
Ask BITMAP for 1 free block
-> gets Block 68
LOG to journal:
'About to write block 68'
Write 'Hello World' to
Block 68 in virtual_disk.bin
Update inode 5:
size=11, blocks=[68]
Update BITMAP:
block 68 = USED
Flush all to disk

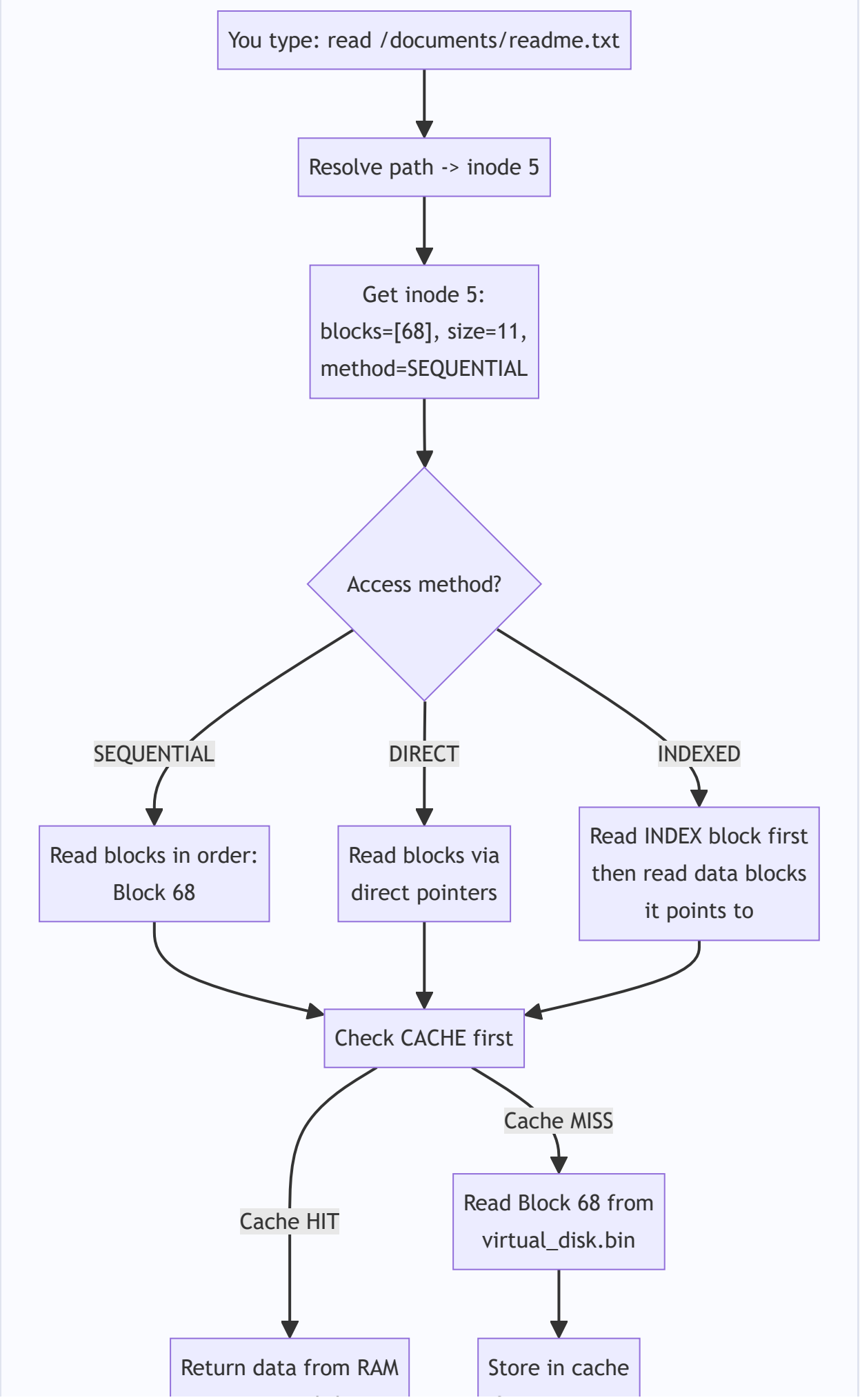


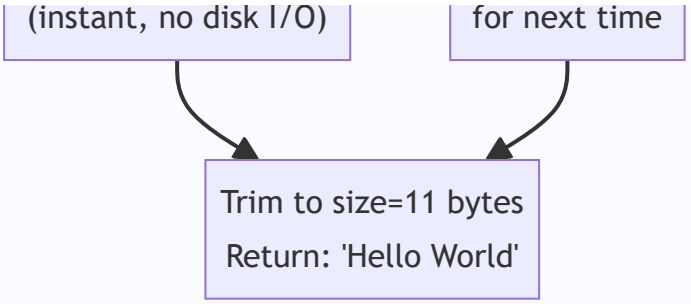
Where is the data physically?

```
virtual_disk.bin:
Byte offset = Block 68 x 4096 = 278,528

At byte 278,528 in the file:
  48 65 6C 6C 6F 20 57 6F 72 6C 64 00 00 00 ...
  H  e  l  l  o     W  o  r  l  d  (padding zeros to fill 4096 bytes)
```

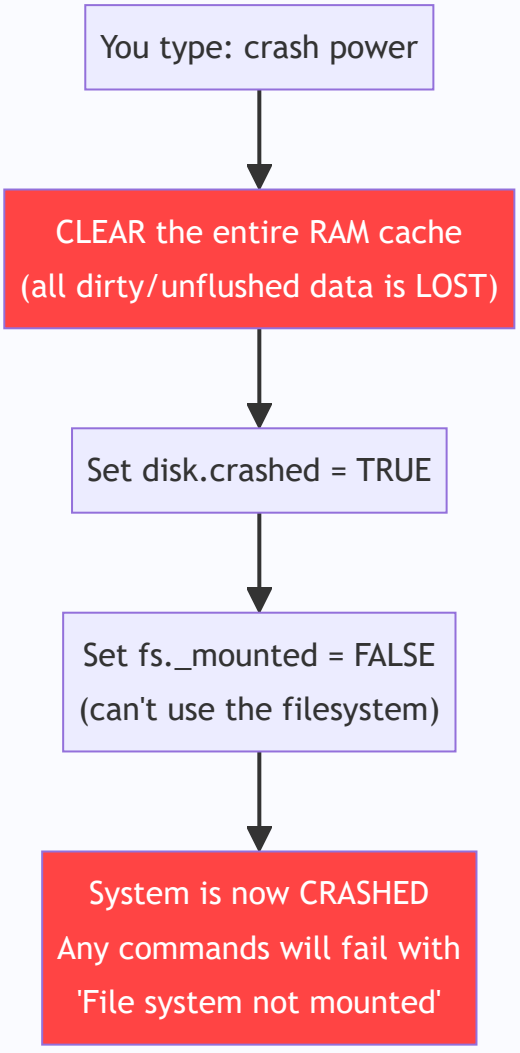
Flow 4: `read /documents/readme.txt` — Reading Data


You type: read /documents/readme.txt
Resolve path -> inode 5
Get inode 5: blocks=[68], size=11, method=SEQUENTIAL
Access method?
SEQUENTIAL
DIRECT
INDEXED
Read blocks in order: Block 68
Read blocks via direct pointers
Read INDEX block first then read data blocks it points to
Check CACHE first
Cache HIT
Cache MISS
Read Block 68 from virtual_disk.bin
Return data from RAM
Store in cache



Flow 5: `crash power` — POWER LOSS SIMULATION

This is the most important demo. Here's what happens:

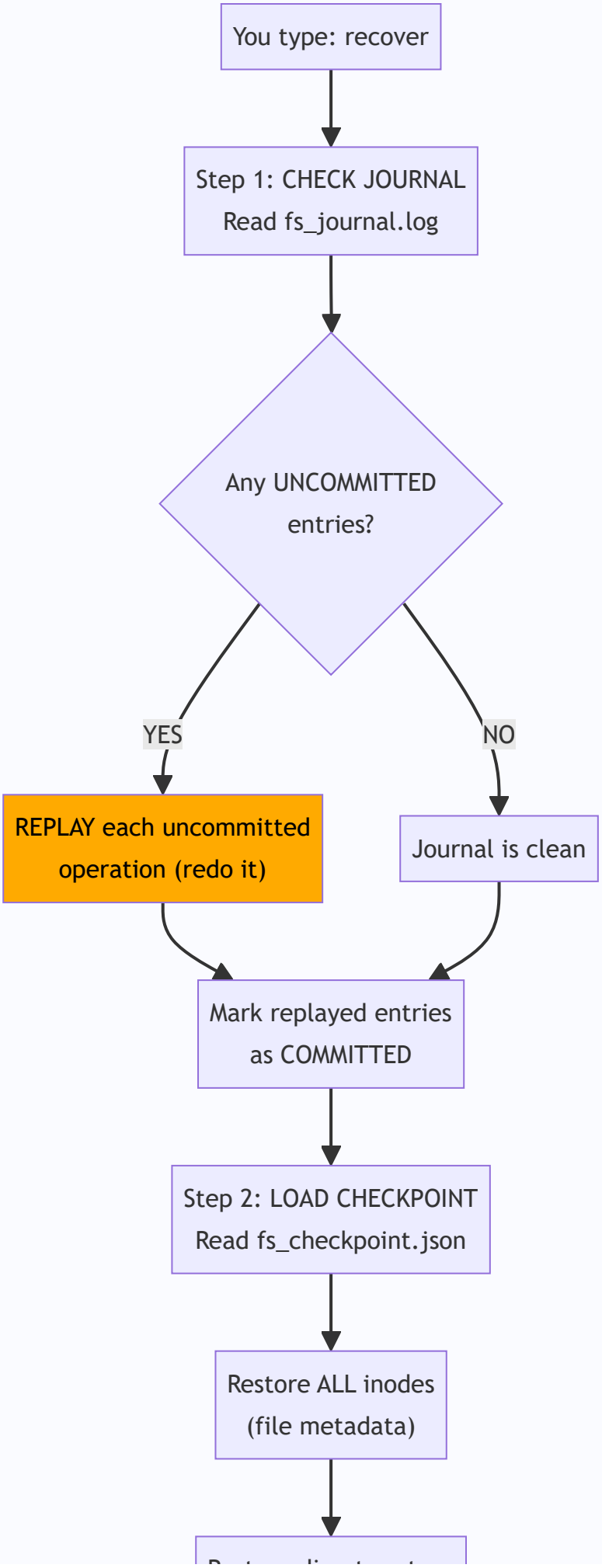


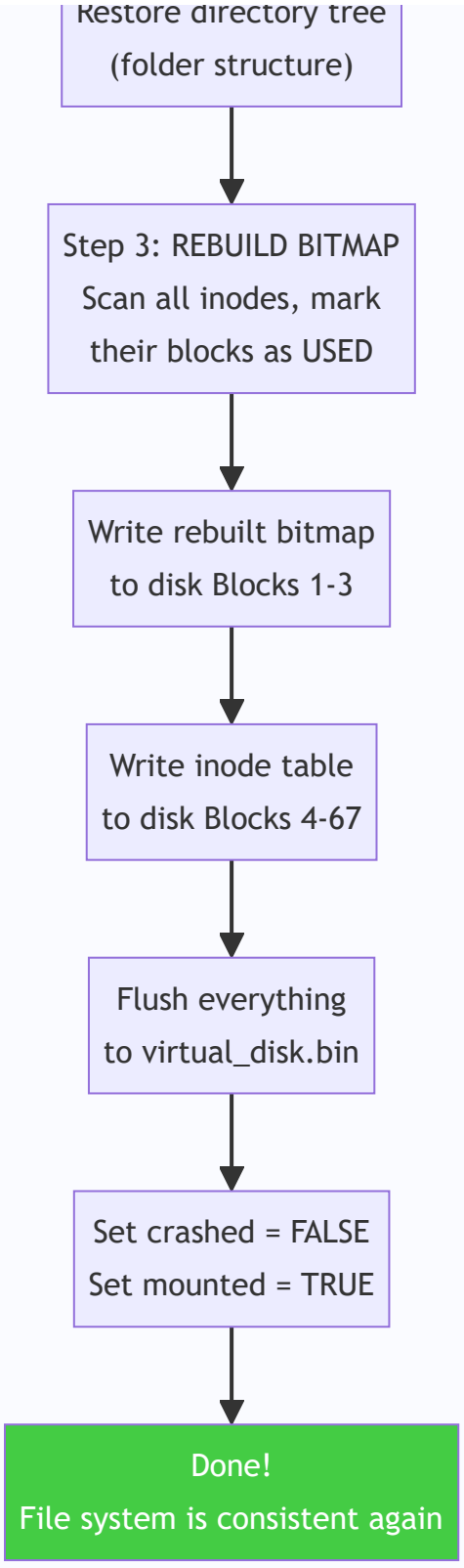
What gets LOST:
- Any data in the cache that wasn't flushed (written to disk)
- Any operations that were logged in journal but not committed

What SURVIVES:

- Everything that was flushed to `virtual_disk.bin`
- The checkpoint file `fs_checkpoint.json` (separate file, untouched)
- The journal file `fs_journal.log` (separate file, untouched)

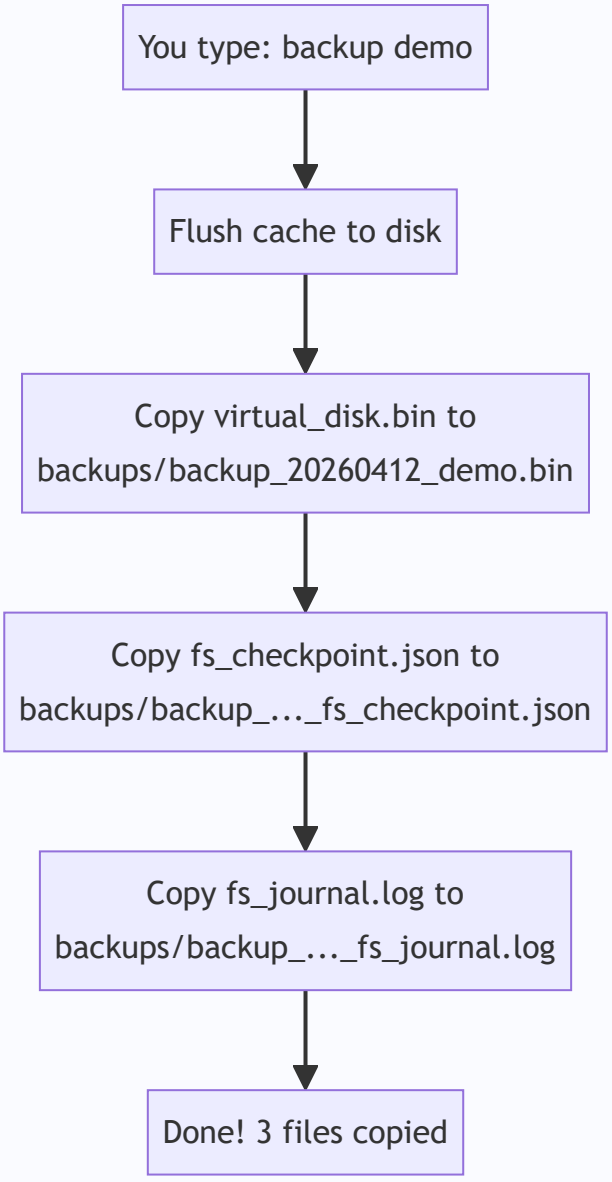
Flow 6: `recover` — CRASH RECOVERY


You type: recover
Step 1: CHECK JOURNAL
Read fs_journal.log
Any UNCOMMITTED entries?
YES
NO
REPLAY each uncommitted operation (redo it)
Journal is clean
Mark replayed entries as COMMITTED
Step 2: LOAD CHECKPOINT
Read fs_checkpoint.json
Restore ALL inodes (file metadata)

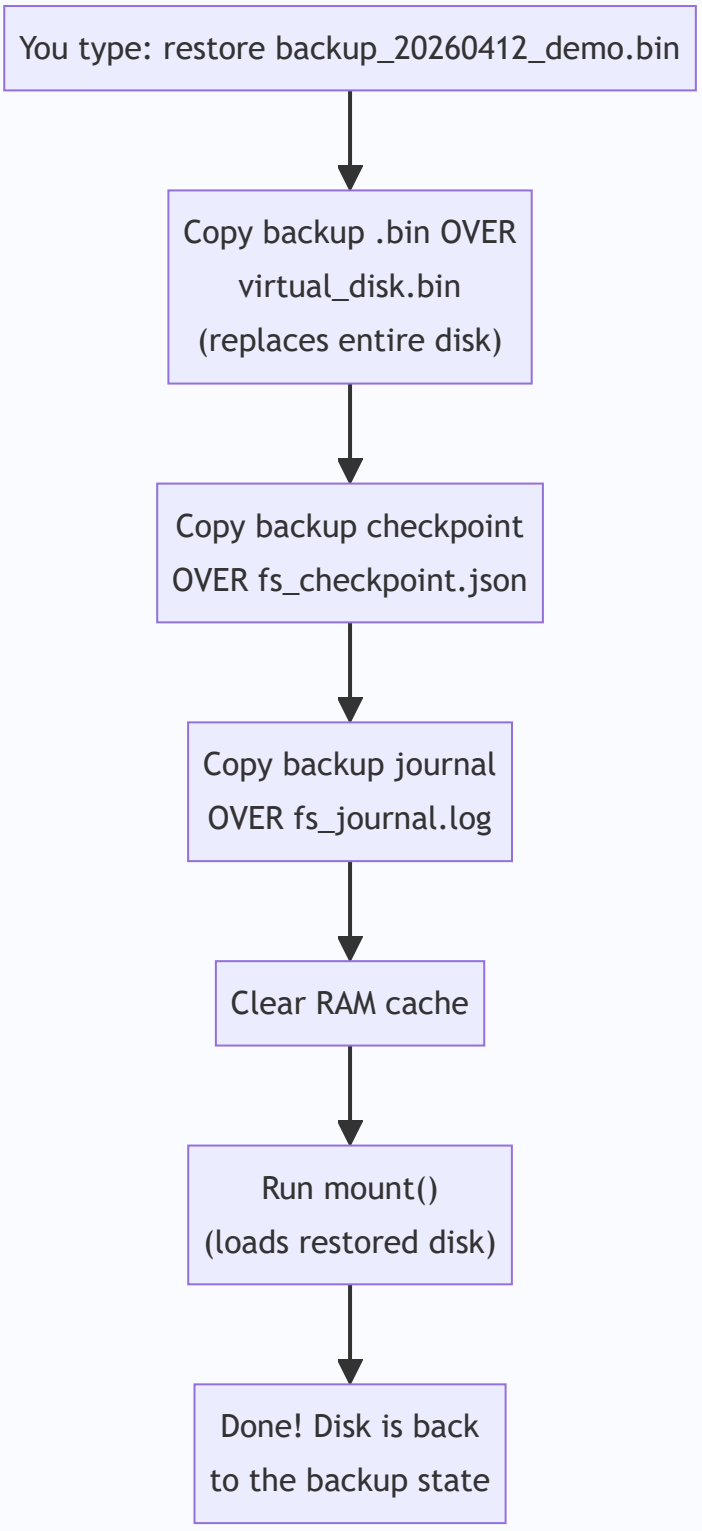


Flow 7: `backup demo` and `restore` — Full Disk Backup

Creating a backup:



Restoring a backup:



Flow 8: `defrag` — Defragmentation

Before defrag (fragmented):

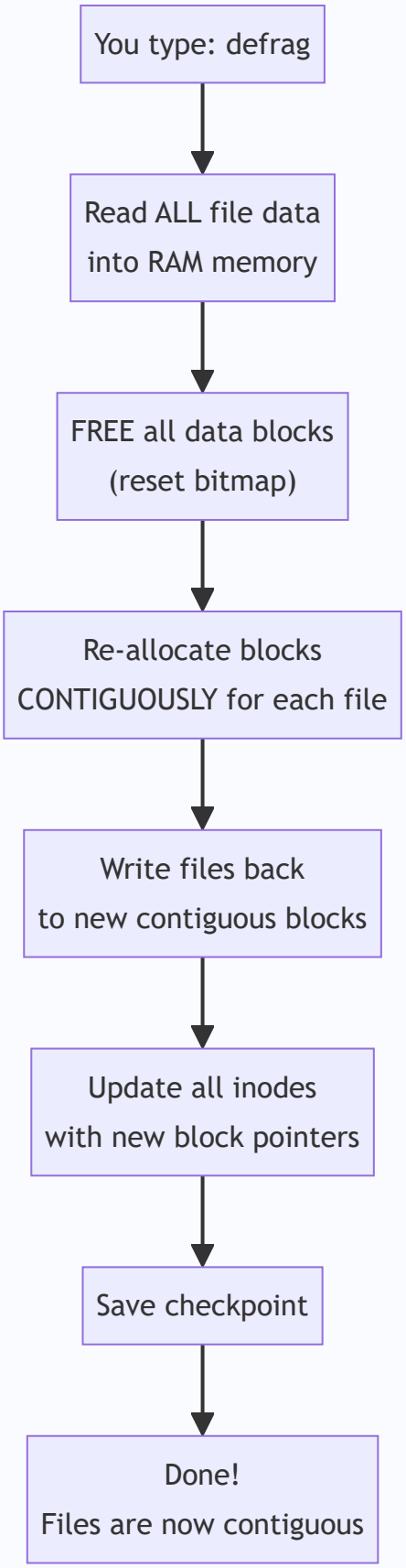
```
Block:  68    69    70    71    72    73    74
Data: [A-p1][FREE][B-p1][A-p2][FREE][B-p2][A-p3]
```

```
File A is in blocks 68, 71, 74 (scattered!)
File B is in blocks 70, 73 (scattered!)
```

After defrag (contiguous):

```
Block:  68    69    70    71    72    73    74
Data: [A-p1][A-p2][A-p3][B-p1][B-p2][FREE][FREE]

File A is in blocks 68, 69, 70 (together!)
File B is in blocks 71, 72 (together!)
```


You type: defrag
Read ALL file data into RAM memory
FREE all data blocks (reset bitmap)
Re-allocate blocks CONTIGUOUSLY for each file
Write files back to new contiguous blocks
Update all inodes with new block pointers
Save checkpoint
Done! Files are now contiguous

Part 4: HOW TO RUN THIS FROM SCRATCH

Step 1: Open PowerShell/Terminal

```
cd "c:\Ashwell\Project\OS Project"
```

Step 2: Run the interactive CLI

```
$env:PYTHONIOENCODING="utf-8"; python cli.py
```

Step 3: Type commands in this order

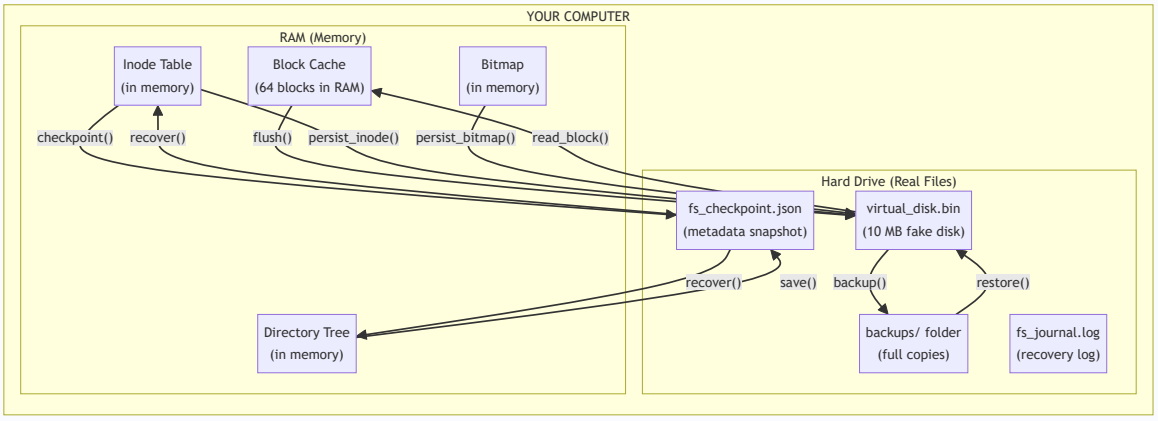
```
fs> format                          ⟵ Creates the virtual disk
fs> mkdir /documents                ⟵ Creates a folder
fs> create /documents/hello.txt     ⟵ Creates an empty file
fs> write /documents/hello.txt Hi   ⟵ Writes "Hi" to the file
fs> read /documents/hello.txt       ⟵ Reads it back: "Hi"
fs> tree                            ⟵ Shows folder structure
fs> metrics                         ⟵ Shows performance stats
fs> checkpoint                      ⟵ Saves metadata snapshot
fs> crash power                     ⟵ SIMULATES POWER LOSS!
fs> recover                         ⟵ RECOVERS FROM CRASH!
fs> read /documents/hello.txt       ⟵ Data is still there!
fs> exit                            ⟵ Quit
```

OR: Run the automated demo

```
$env:PYTHONIOENCODING="utf-8"; python test_simulation.py
```

This runs all 13 test stages automatically with no typing needed.

Part 5: COMPLETE DATA FLOW DIAGRAM (Everything Together)



Summary Table: Every File At a Glance

File	Type	Size	Purpose	Created When
`disk.py`	Source code	12 KB	Simulates the hard disk, cache, crash functions	You wrote it
`free_space_manager.py`	Source code	8 KB	Tracks free/used blocks (bitmap)	You wrote it
`inode.py`	Source code	6 KB	Stores file metadata (name, size, blocks)	You wrote it
`directory.py`	Source code	9 KB	Manages folder hierarchy tree	You wrote it
`file_system.py`	Source code	32 KB	The brain connecting everything	You wrote it
`recovery.py`	Source code	10 KB	Journal, checkpoint, backup logic	You wrote it

File	Type	Size	Purpose	Created When
`cli.py`	Source code	13 KB	Terminal interface for commands	You wrote it
`test_simulation.py`	Source code	14 KB	Automated 13-step test	You wrote it
`virtual_disk.bin`	Binary	10 MB	THE fake hard disk	`format` command
`fs_journal.log`	Text/JSON	~1 KB	Operation log for crash recovery	First file operation
`fs_checkpoint.json`	JSON	~5 KB	Full metadata snapshot	`checkpoint` / `format`
`backups/`	Folder	~10 MB each	Full disk copies	`backup` command